

阿里云高级前端开发（AI经验）

一面（技术基础） · 1-6

1. 请简述一下浏览器的事件循环机制，特别是微任务和宏任务的执行顺序。
2. 在 AI 对话场景中，SSE（Server-Sent Events）和 WebSocket 有什么区别？你如何选择？
3. TypeScript 中的泛型约束（extends）和类型断言（as）分别在什么场景下使用？
4. 手写代码：实现一个简单的 Promise.all 方法。
5. CSS 中实现一个打字机效果的动画，模拟 AI 字符输出。
6. 什么是 Prompt Engineering（提示工程）？在前端开发中如何优化 Prompt？

二面（深入原理与项目） · 1-8

1. 谈谈 React Fiber 架构解决了什么问题？它的工作原理是怎样的？
2. 在 AI 聊天应用中，如何处理 Markdown 渲染和代码高亮的性能问题？
3. 请解释 RAG（检索增强生成）的前端参与流程及其核心价值。
4. 如何实现一个 AI Agent 的 Function Calling（函数调用）流程？
5. 你们项目里是如何做前端性能优化的？请结合 AI 场景谈谈。
6. 实现一个深拷贝函数，需要考虑循环引用和特类型（如 Map, Set）。
7. 在大模型应用中，如何处理 Token 长度限制带来的上下文丢失问题？
8. 谈谈微前端架构（如 qiankun）的实现原理，它在大型 AI 平台中有什么作用？

三面（架构与综合能力） · 1-5

1. 如果让你从零设计一个类似 ChatGPT 的企业级 AI 助手前端架构，你会考虑哪些核心模块？
2. 在 AI 赋能前端开发的趋势下，你认为前端工程师的核心竞争力会发生什么变化？
3. 聊聊你在过去项目中遇到的最难的一个技术问题，你是如何解决的？
4. 如何评价一个 AI 产品的“前端交互好坏”？有哪些具体的衡量指标？
5. 面对快速迭代的 AI 技术，你平时是如何保持技术领先并将其落地到业务中的？

一面（技术基础）

1. 请简述一下浏览器的事件循环机制，特别是微任务和宏任务的执行顺序。

难度：★

考点：浏览器原理、异步执行机制

答案要点：

事件循环是浏览器协调事件、用户交互、脚本、渲染、网络等任务的机制，它确保了单线程的 JS 能够非阻塞运行。

- 执行栈清空后，优先处理所有微任务队列中的任务。
- 微任务处理完后，检查是否需要渲染更新，然后从宏任务队列取出一个任务执行。
- 常见的宏任务包括 setTimeout、setInterval、I/O、UI 渲染。
- 常见的微任务包括 Promise.then、MutationObserver、queueMicrotask。

常见坑：

- 误以为 Promise 构造函数内部是异步的（实际上是同步执行）。
- 忽视了 await 后面的代码会被包装成微任务。

追问：

- 如果在微任务中不断产生新的微任务，会发生什么？

2. 在 AI 对话场景中，SSE（Server-Sent Events）和 WebSocket 有什么区别？你如何选择？

难度：★★

考点：网络协议、流式传输

答案要点：

SSE 是基于 HTTP 的单向流式传输协议，而 WebSocket 是全双工的 TCP 连接，AI 问答通常首选 SSE。

- SSE 更加轻量，支持自动重连，且天然支持 HTTP 代理和负载均衡。
- WebSocket 支持双向通信，适合实时协作或游戏场景，但维护成本和协议开销较高。
- AI 场景多为“一问一答”且回复较长，SSE 的单向流特性完美契合大模型输出。
- SSE 只能传输文本，若需传输二进制数据，WebSocket 或 Fetch 流更合适。

常见坑：

- 忽略了 SSE 在 HTTP/1.1 下有 6 个并发连接限制的问题。
- 没考虑到 SSE 无法通过 POST 请求发送大数据（标准 SSE 仅支持 GET）。

追问：

- 如何在 SSE 中实现类似 POST 的大数据提交？

3. TypeScript 中的泛型约束（extends）和类型断言（as）分别在什么场景下使用？

难度：★★

考点：TS 基础、类型安全

答案要点：

泛型约束用于限制泛型参数必须符合某种结构，而类型断言则是开发者手动告诉编译器变量的实际类型。

- 泛型约束：通过 `T extends K` 确保传入的类型具备特定的属性或方法。
- 类型断言：在编译器无法推断出精确类型，但开发者明确知道类型时使用，如处理 API 返回值。
- 泛型约束是在定义阶段增强代码健壮性，类型断言是在使用阶段规避编译错误。
- 尽量减少使用 `as any`，应优先使用 `as Type` 或类型守卫。

常见坑：

- 滥用类型断言导致运行时报错，因为断言并不会进行实际的类型转换。
- 混淆了 `extends` 在条件类型和泛型约束中的不同含义。

追问：

- 什么是 `const` 断言，它有什么作用？

4. 手写代码：实现一个简单的 `Promise.all` 方法。

难度：★★

考点：手写代码、异步并发控制

答案要点：

`Promise.all` 接收一个 `Promise` 数组，只有当所有 `Promise` 都成功时才返回成功结果数组，否则返回第一个失败的结果。

- 返回一个新的 `Promise` 实例。
- 维护一个结果数组和一个计数器，确保结果顺序与输入顺序一致。
- 遍历输入数组，对每个项使用 `Promise.resolve` 包装以兼容非 `Promise` 值。
- 计数器达到数组长度时，调用 `resolve`。

常见坑：

- 直接使用 `results.length === args.length` 判断结束（可能会因为空位导致错误）。
- 忘记处理输入参数不是数组的情况。

代码示例：

代码块

```
1 function promiseAll(promises) {
2   return new Promise((resolve, reject) => {
3     if (!Array.isArray(promises)) return reject(new TypeError('Arguments must
    be an array'));
4     let resolvedCount = 0;
5     const results = [];
6     promises.forEach((p, index) => {
7       Promise.resolve(p).then(res => {
```

```
8         results[index] = res;
9         resolvedCount++;
10        if (resolvedCount === promises.length) resolve(results);
11    }, reject);
12    });
13    });
14 }
```

追问：

- 如果想实现“无论成功失败都返回所有结果”，应该用哪个 API？

5. CSS 中实现一个打字机效果的动画，模拟 AI 字符输出。

难度：★

考点：CSS 动画、用户体验

答案要点：

可以通过设置容器宽度从 0 到 100% 的变化，结合 `steps()` 函数和 `overflow: hidden` 来实现。

- 使用 `keyframes` 定义宽度变化。
- `animation-timing-function` 使用 `steps(n)`，n 为字符个数，产生跳跃感。
- 设置 `white-space: nowrap` 防止换行破坏效果。
- 利用右边框（`border-right`）模拟闪烁的光标。

常见坑：

- 字符宽度不一致导致动画对不齐（建议使用等宽字体 `monospace`）。
- 动态内容长度不固定时，静态 CSS 较难处理。

追问：

- 对于 AI 实时生成的长文本，CSS 动画还适用吗？如果不适用该怎么做？

6. 什么是 Prompt Engineering（提示工程）？在前端开发中如何优化 Prompt？

难度：★★

考点：AI 基础、Prompt 优化

答案要点：

提示工程是通过精心设计输入文本，引导大模型输出更准确、更符合预期结果的技术。

- 角色设定：明确告诉模型它是一个“资深前端专家”或“翻译官”。
- 少样本提示（Few-shot）：提供几个输入输出示例，让模型模仿。
- 结构化输出：要求模型返回 JSON 格式，方便前端解析。

- 任务拆解：将复杂问题拆分为多个子步骤（Chain of Thought）。

常见坑：

- Prompt 过长导致消耗过多的 Token 且模型注意力分散。
- 忽略了负向约束（Negative Prompt），即明确告诉模型不要输出什么。

追问：

- 如何防止用户在输入框中通过 Prompt 注入攻击你的系统？
-

二面（深入原理与项目）

1. 谈谈 React Fiber 架构解决了什么问题？它的工作原理是怎样的？

难度：★★★

考点：框架深度原理、性能调度

答案要点：

Fiber 主要是为了解决 React 在处理大型组件树时，由于同步递归更新导致的浏览器卡顿问题。

- 将更新任务拆分为小片（Fiber 节点），支持任务的优先级、中断和恢复。
- 双缓存技术：在内存中构建 workInProgress tree，完成后一次性提交到真实 DOM。
- 两个阶段：Render 阶段（可中断，计算差异）和 Commit 阶段（不可中断，操作 DOM）。
- 引入了 Scheduler 调度器，利用浏览器空闲时间（requestIdleCallback 思想）执行任务。

常见坑：

- 误认为 Fiber 提高了单次计算的速度（实际上增加了开销，但提升了响应性）。
- 在 Render 阶段执行了具有副作用的操作。

追问：

- 为什么 React 团队要自己实现 Scheduler 而不直接用 requestIdleCallback？

2. 在 AI 聊天应用中，如何处理 Markdown 渲染和代码高亮的性能问题？

难度：★★

考点：性能优化、第三方库应用

答案要点：

AI 输出是流式的，频繁的 Markdown 解析和语法高亮会导致严重的 CPU 占用和界面闪烁。

- 增量渲染：只对新产生的片段进行解析，或使用带有缓存机制的解析器（如 markdown-it）。
- 防抖/节流渲染：不要每出一个字符就渲染一次，而是按时间片（如 100ms）更新。
- Web Worker：将复杂的 Markdown 转 HTML 逻辑移出主线程。
- 虚拟列表：当对话历史极长时，只渲染可视区域内的消息卡片。

常见坑：

- 每次流更新都重新创建整个 DOM 树，导致失去焦点或滚动位置丢失。
- 代码高亮库（如 Prism.js）在处理超长代码块时阻塞主线程。

追问：

- 如何实现流式输出时的“自动滚动到底部”功能，且不干扰用户手动向上滚动？

3. 请解释 RAG（检索增强生成）的前端参与流程及其核心价值。

难度：★★★

考点：AI 架构、RAG

答案要点：

RAG 通过在生成答案前从私有知识库检索相关信息，解决了大模型幻觉和时效性问题，前端主要负责交互和上下文呈现。

- 核心流程：用户提问 -> 向量化检索 -> 拼接 Context -> 提交 LLM -> 获取答案。
- 前端职责：处理文档上传、切片预览、展示引用来源（Citations）以及反馈纠错。
- 价值：让模型能够回答企业内部或最新的知识，降低模型训练成本。
- 交互设计：点击引用标注时，前端需要精准跳转并高亮原始文档的对应片段。

常见坑：

- 忽略了检索结果的排序对最终生成质量的影响。
- 前端展示引用来源时，没有处理好长文本截断和溯源链接的有效性。

追问：

- 如果检索出来的上下文太长超过了 Token 限制，前端或后端通常怎么处理？

4. 如何实现一个 AI Agent 的 Function Calling（函数调用）流程？

难度：★★★

考点：AI Agent、交互逻辑

答案要点：

Function Calling 允许模型根据用户意图调用外部 API，实现从“对话”到“执行”的跨越。

- 定义 Schema：前端或后端向模型提供工具函数的名称、参数描述（JSON Schema）。
- 模型决策：模型判断是否需要调用工具，并返回符合 Schema 的参数。
- 本地执行：前端或后端根据模型返回的参数执行实际代码（如查询天气、绘图）。
- 结果回传：将执行结果再次发送给模型，由模型总结并回复用户。

常见坑：

- 模型生成的参数不符合 JSON 格式或类型错误，需要前端做容错校验。
- 循环调用问题：模型可能陷入无意义的连续函数调用中。

追问：

- 在前端执行 Function Calling 时，如何保证安全性？

5. 你们项目里是如何做前端性能优化的？请结合 AI 场景谈谈。

难度：★★

考点：性能优化、工程实践

答案要点：

除了常规的资源压缩和 CDN，AI 场景更关注长列表渲染、流式响应速度和内存管理。

- 资源加载：对 heavy 的 AI 相关库（如 MathJax, Monaco Editor）进行异步加载和分包。
- 渲染优化：使用 React.memo 或 Vue 的 shallowRef 减少不必要的对话框重绘。
- 内存清理：及时销毁不再需要的流监听器和长连接，防止内存泄漏。
- 预加载：在用户输入时预热连接或预加载可能用到的模型配置。

常见坑：

- 盲目使用 SSR，导致流式输出在首屏渲染时出现水合（Hydration）错误。
- 忽略了移动端在处理大量 AI 生成内容时的发热和掉帧问题。

追问：

- 如何监控 AI 响应的“首字延迟（TTFT）”并进行针对性优化？

6. 实现一个深拷贝函数，需要考虑循环引用和特殊类型（如 Map, Set）。

难度：★★

考点：JS 基础、递归、数据结构

答案要点：

深拷贝需要递归遍历对象，并处理各种边缘情况，最核心的是解决循环引用导致的死循环。

- 使用 `WeakMap` 存储已拷贝的对象，遇到重复引用直接返回。
- 区分处理普通对象、数组、Map、Set、Date、RegExp 等类型。
- 考虑 Symbol 属性的拷贝。
- 递归时注意栈溢出风险。

代码示例：

代码块

```
1 function deepClone(obj, map = new WeakMap()) {
2   if (obj === null || typeof obj !== 'object') return obj;
3   if (obj instanceof Date) return new Date(obj);
4   if (obj instanceof RegExp) return new RegExp(obj);
5   if (map.has(obj)) return map.get(obj);
6
7   const cloneObj = new obj.constructor();
8   map.set(obj, cloneObj);
9
10  if (obj instanceof Map) {
```

```
11     obj.forEach((val, key) => cloneObj.set(deepClone(key, map), deepClone(val,
    map)));
12   } else if (obj instanceof Set) {
13     obj.forEach(val => cloneObj.add(deepClone(val, map)));
14   } else {
15     Reflect.ownKeys(obj).forEach(key => {
16       cloneObj[key] = deepClone(obj[key], map);
17     });
18   }
19   return cloneObj;
20 }
```

追问：

- 为什么这里使用 WeakMap 而不是 Map？

7. 在大模型应用中，如何处理 Token 长度限制带来的上下文丢失问题？

难度：★★

考点：LLM 限制、状态管理

答案要点：

由于模型窗口有限，必须采取策略在保留核心信息的同时精简上下文。

- 滑动窗口：只保留最近的 N 轮对话记录。
- 摘要压缩：将较早的对话通过模型生成一份简短摘要，替换掉原始长文本。
- 关键信息提取：仅保留用户画像、当前任务目标等核心 Metadata。
- 向量检索：将历史记录存入向量数据库，仅在需要时按需检索相关片段。

常见坑：

- 简单粗暴地截断文本，导致模型丢失关键的指令或前提条件。
- 忽略了 System Prompt 也占用 Token 空间。

追问：

- 前端如何预估当前输入的 Token 数量？

8. 谈谈微前端架构（如 qiankun）的实现原理，它在大型 AI 平台中有什么作用？

难度：★★★

考点：架构设计、微前端

答案要点：

微前端通过将巨型应用拆分为多个独立运行的子应用，解决了团队协作和技术栈隔离的问题。

- 路由劫持：通过监听 hashchange 或 popstate 事件，匹配并加载对应子应用。
- 应用加载：通过 fetch 获取子应用 HTML，解析出 JS/CSS 并动态执行。

- 沙箱隔离：使用 Proxy 实现 Window 对象的快照或代理，防止全局变量污染。
- 样式隔离：通过 Shadow DOM 或 CSS 前缀（Scoped CSS）防止样式冲突。

常见坑：

- 子应用中的全局定时器或事件监听在卸载时未清理。
- 多个子应用同时调用 AI 接口时的鉴权和状态共享问题。

追问：

- 微前端模式下，如何实现主子应用之间的通信？

三面（架构与综合能力）

1. 如果让你从零设计一个类似 ChatGPT 的企业级 AI 助手前端架构，你会考虑哪些核心模块？

难度：★★★

考点：系统设计、全局视野

答案要点：

一个成熟的 AI 前端架构需要兼顾交互体验、性能稳定性和 AI 特性的快速迭代。

- 通信层：支持 SSE/WebSocket/Fetch 流式协议，具备完善的重连和错误处理机制。
- 渲染引擎：高性能 Markdown 解析、多模态支持（图片、文件、图表）、代码沙箱。
- 状态管理：对话树管理（支持分支对话）、上下文 Token 计数、历史记录持久化。
- 插件/工具系统：支持 Function Calling 的 UI 动态注入和交互反馈。
- 安全与监控：Prompt 注入防御、敏感词过滤、用户行为分析、性能指标监控。

常见坑：

- 架构过于耦合，导致更换底层模型或通信协议时需要大规模重构。
- 忽视了多端适配（PC/移动端/插件端）的逻辑复用。

2. 在 AI 赋能前端开发的趋势下，你认为前端工程师的核心竞争力会发生什么变化？

难度：★★

考点：行业思考、职业规划

答案要点：

AI 将替代重复性的搬砖工作，前端工程师将从“代码编写者”转向“系统编排者”和“用户体验设计师”。

- 熟练使用 AI 工具（Copilot, Cursor）提升开发效率，关注如何写出更好的 Prompt。
- 深入理解 AI 原理，能够参与 RAG 优化、Agent 设计等跨界工作。

- 更加关注复杂交互设计和极致性能优化，这些是 AI 目前难以完全替代的。
- 具备更强的业务理解能力，能够利用 AI 解决具体的业务痛点。

常见坑：

- 产生焦虑感而忽视了对前端基础知识（JS/浏览器原理）的深耕。
- 过度依赖 AI 生成代码而失去了代码审查和 Debug 的能力。

3. 聊聊你在过去项目中遇到的最难的一个技术问题，你是如何解决的？

难度：★★★

考点：解决问题能力、技术深度

答案要点：

（此题为开放题，需结合实际 AI 项目经验回答）

- 描述背景：例如在处理超长流式文本渲染时出现的掉帧问题。
- 分析过程：使用 Performance 面板定位瓶颈，发现是大量 DOM 操作和样式计算。
- 解决方案：引入了虚拟列表、分片渲染以及 Web Worker 处理解析逻辑。
- 最终结果：量化指标（如 FPS 从 20 提升到 60，首屏时间降低 50%）。

常见坑：

- 讲得太笼统，缺乏具体的技术细节和数据支撑。
- 解决方案听起来太常规，体现不出“高级”岗位的思考深度。

4. 如何评价一个 AI 产品的“前端交互好坏”？有哪些具体的衡量指标？

难度：★★

考点：产品意识、用户体验

答案要点：

AI 产品的交互核心在于“消除不确定性”和“提升反馈效率”。

- 响应速度：首字生成延迟（TTFT）、整体生成速度（Tokens/s）。
- 确定性反馈：是否有清晰的 Loading 状态、思考过程展示、流式动画是否平滑。
- 容错处理：当模型输出错误或中断时，是否有重试、编辑、反馈（点赞/点踩）机制。
- 易用性：Prompt 模板是否丰富、历史记录搜索是否便捷、多模态输入是否自然。

常见坑：

- 只关注 UI 美观度，忽略了 AI 响应慢带来的焦虑感。
- 缺乏对模型异常情况（如幻觉、断线）的交互预案。

5. 面对快速迭代的 AI 技术，你平时是如何保持技术领先并将其落地到业务中的？

难度：★★

考点：学习能力、技术落地

答案要点：

保持敏锐的技术嗅觉，并建立从“输入”到“产出”的闭环。

- 持续关注前沿：阅读 Arxiv 论文、关注 LangChain/Vercel AI SDK 等开源社区动态。
- 动手实践：通过 Side Project 尝试新工具（如 AutoGPT, Vector DB）。
- 业务结合：主动思考 AI 如何优化现有业务流程（如自动化测试、低代码生成）。
- 知识沉淀：在团队内分享 AI 技术，推动团队整体 AI 化转型。

常见坑：

- 只是“看”而不“写”，对技术的理解停留在表面。
- 盲目追新，在不成熟的业务场景中强行引入复杂的 AI 方案。